# Prelucrarea Imaginilor

Prof. Dr. Călin-Adrian POPA

## Cursul 5

27 Octombrie 2025

# 5 Attention mechanisms

- the optic nerve of our visual system receives massive sensory input, far exceeding what the brain can fully process
- fortunately, not all stimuli are equal
- focalization and concentration of consciousness have enabled us to direct attention to objects of interest, such as preys and predators, in the complex visual environment
- the ability of paying attention to only a small fraction of the information has evolutionary significance, allowing human beings to live and succeed

# 5.1 Attention cues

- information in our environment is not scarce, attention is
- when inspecting a visual scene, our optic nerve receives information at the order of $10^8$ bits per second, far exceeding what our brain can fully process
- fortunately, we learned from experience (also known as data) that *not all sensory inputs are equal*
- throughout human history, the capability of directing attention to only a fraction of information of interest has enabled our brain to allocate resources more smartly to survive, to grow, and to socialize, such as detecting predators, preys, and mates

# 5.1 Attention cues

- to explain how our attention is deployed in the visual world, a two-component framework has emerged
- in this framework, subjects selectively direct the spotlight of attention using both the *nonvolitional cue* and the *volitional cue*
- the nonvolitional cue is based on the *saliency* of objects in the environment
- imagine there are five objects in front of us: a newspaper, a research paper, a cup of coffee, a notebook, and a book, such as in Figure 1
- while all the paper products are printed in black and white, the coffee cup is red
- in other words, this coffee is intrinsically salient in this visual environment, automatically and involuntarily drawing attention
- so, we bring the fovea (the center of the macula, where visual acuity is highest) onto the coffee, as shown in Figure 1

# 5.1 Attention cues

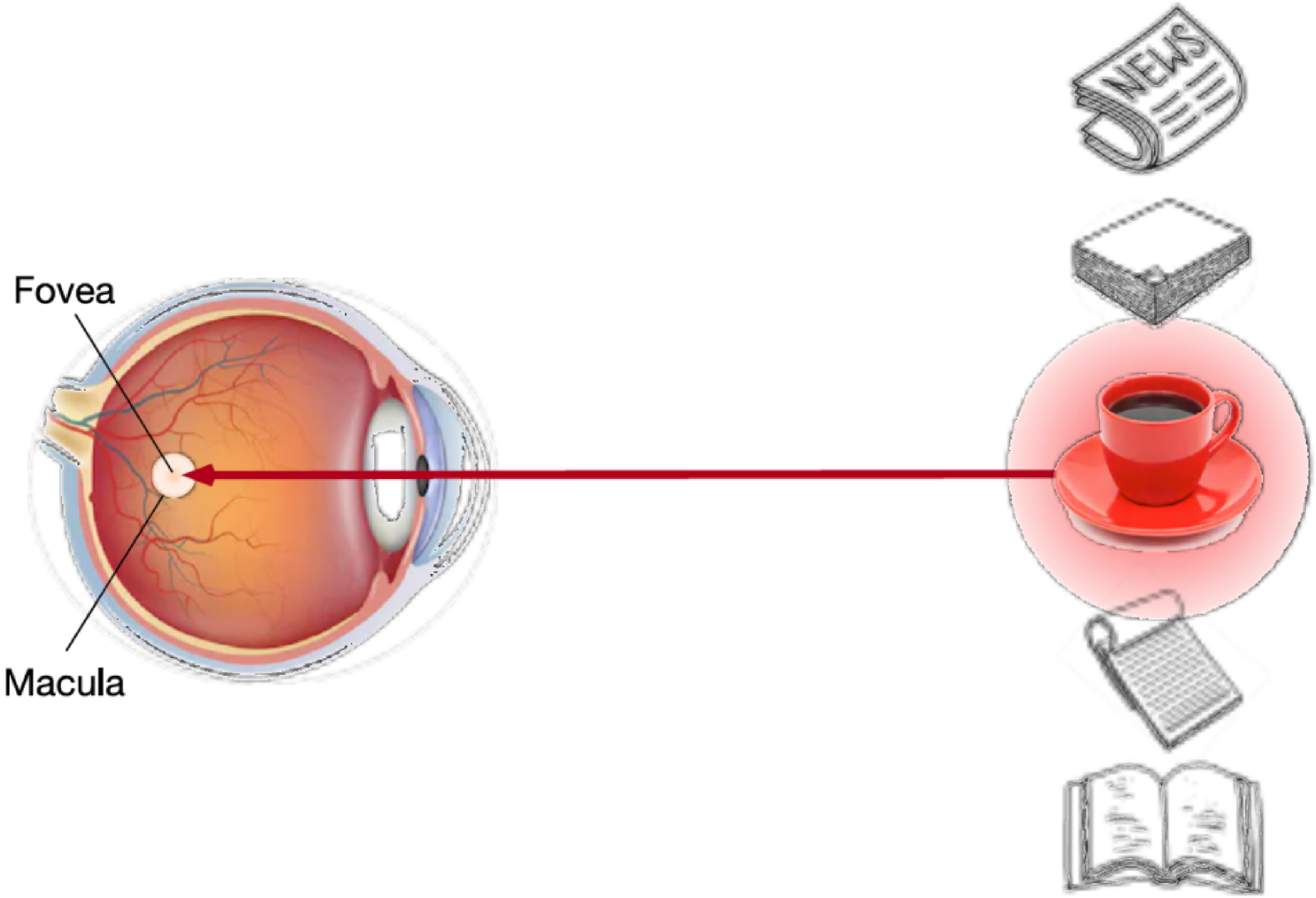



Figure 1: Using the nonvolitional cue based on saliency (red cup, non-paper), attention is involuntarily directed to the coffee.

# 5.1 Attention cues

- after drinking coffee, we become caffeinated, and want to read a book
- so we turn our head, refocus our eyes, and look at the book, as depicted in Figure 2
- different from the case in Figure 1, where the coffee biases us towards selecting based on saliency, in this task-dependent case, we select the book under cognitive and volitional control
- using the volitional cue based on variable selection criteria, this form of attention is more deliberate
- it is also more powerful, with the subject's voluntary effort

# 5.1 Attention cues

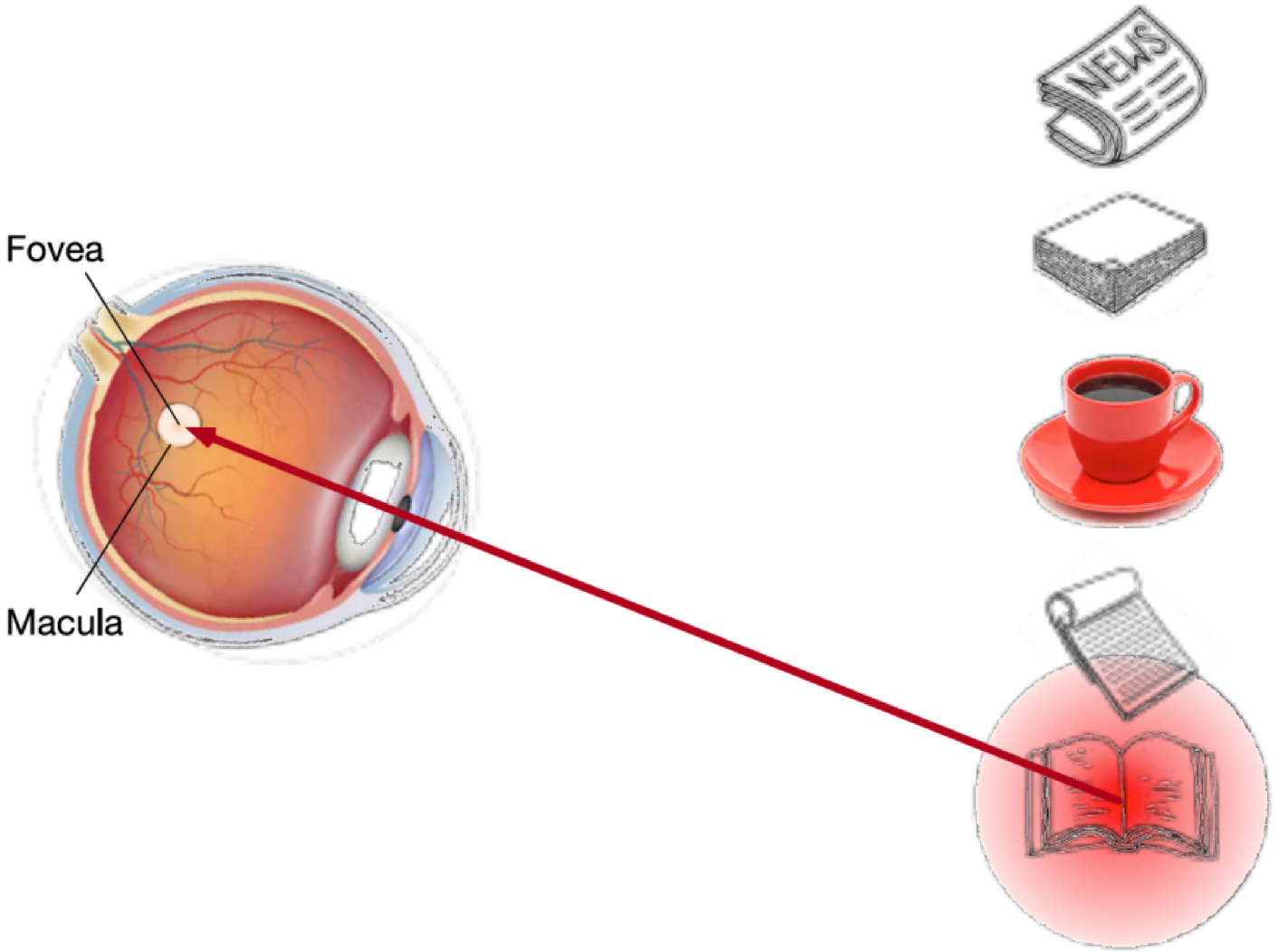



Figure 2: Using the volitional cue (want to read a book) that is task-dependent, attention is directed to the book under volitional control.

# 5.1 Attention cues

- inspired by the nonvolitional and volitional attention cues that explain the attentional deployment, in the following we will describe a framework for designing attention mechanisms by incorporating these two attention cues
- to begin with, consider the simpler case where only nonvolitional cues are available
- to bias selection over sensory inputs, we can simply use a parameterized fully-connected layer or even a non-parameterized max or average pooling layer

# 5.1 Attention cues

- therefore, what sets attention mechanisms apart from those fully-connected layers or pooling layers is the inclusion of the volitional cues
- in the context of attention mechanisms, we refer to volitional cues as *queries*
- given any query, attention mechanisms bias selection over sensory inputs (e.g., intermediate feature representations) via *attention pooling*
- these sensory inputs are called *values*, in the context of attention mechanisms
- more generally, every value is paired with a *key*, which can be thought of as the nonvolitional cue of that sensory input
- as shown in Figure 3, we can design attention pooling so that the given query (volitional cue) can interact with keys (nonvolitional cues), which guides bias selection over values (sensory inputs)

# 5.1 Attention cues

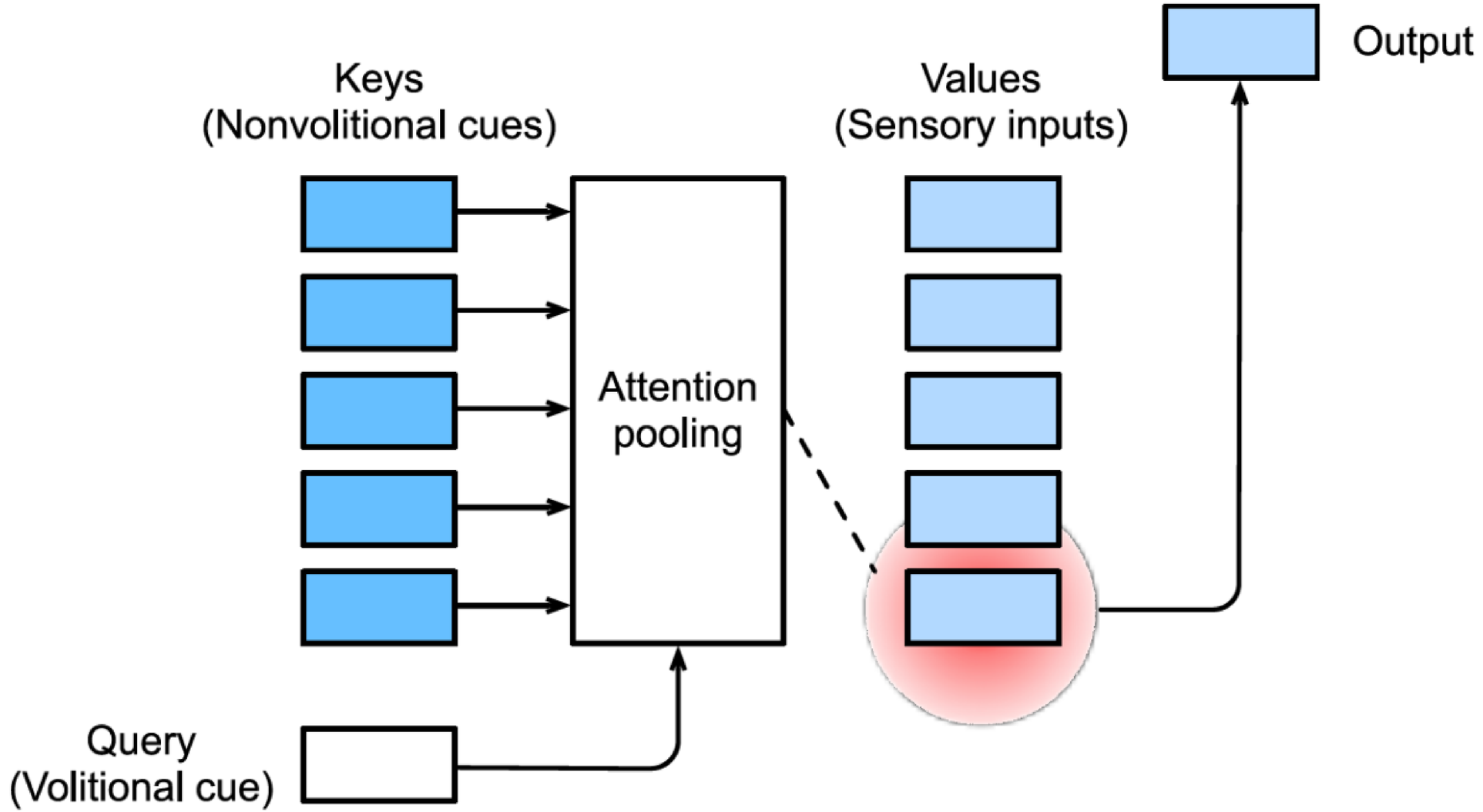



Figure 3: Attention mechanisms bias selection over values (sensory inputs) via attention pooling, which incorporates queries (volitional cues) and keys (nonvolitional cues).

# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- now we know the major components of attention mechanisms under the framework in Figure 3
- to recapitulate, the interactions between queries (volitional cues) and keys (nonvolitional cues) result in *attention pooling*
- the attention pooling selectively aggregates values (sensory inputs) to produce the output
- in this section, we will describe attention pooling in greater detail, to give a high-level view of how attention mechanisms work in practice
- specifically, the Nadaraya-Watson kernel regression model, proposed in 1964, is a simple yet complete example for demonstrating machine learning with attention mechanisms

# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- to keep things simple, let us consider the following regression problem: given a dataset of input-output pairs $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, how to learn $f$ to predict the output $\hat{y} = f(x)$ for any new input $x$?
- we begin with perhaps the world's "dumbest" estimator for this regression problem: using average pooling to average over all the training outputs:

$$f(x) = \frac{1}{n} \sum_{i=1}^{n} y_i. \tag{1}$$

- obviously, average pooling omits the inputs $x_i$
- a better idea was proposed by Nadaraya and Watson, namely to weigh the outputs $y_i$ according to their input locations:

$$f(x) = \sum_{i=1}^{n} \frac{K(x - x_i)}{\sum_{j=1}^{n} K(x - x_j)} y_i, \tag{2}$$

  where $K$ is a *kernel*
- the estimator in (2) is called *Nadaraya-Watson kernel regression*; here, we will not dive into details of kernels

# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- recall the framework of attention mechanisms in Figure 3; from the perspective of attention, we can rewrite (2) in a more generalized form of *attention pooling*:

$$f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i, \tag{3}$$

  where $x$ is the *query* and $(x_i, y_i)$ is the *key-value* pair
- comparing (3) and (1), the attention pooling here is a weighted average of values $y_i$
- the *attention weight* $\alpha(x, x_i)$ in (3) is assigned to the corresponding value $y_i$ based on the interaction between the query $x$ and the key $x_i$, modeled by $\alpha$
- for any query, its attention weights over all the key-value pairs are a valid probability distribution: they are non-negative and sum up to one

# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- to gain intuitions of attention pooling, we can consider a Gaussian kernel defined as:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right).$$

- plugging the Gaussian kernel into (3) and (2) gives:

$$\begin{aligned} f(x) &= \sum_{i=1}^{n} \alpha(x, x_i) y_i \\ &= \sum_{i=1}^{n} \frac{\exp\left(-\frac{1}{2}(x - x_i)^2\right)}{\sum_{j=1}^{n} \exp\left(-\frac{1}{2}(x - x_j)^2\right)} y_i \\ &= \sum_{i=1}^{n} \text{softmax}\left(-\frac{1}{2}(x - x_i)^2\right) y_i. \end{aligned} \tag{4}$$

- in (4), a key $x_i$ that is closer to the given query $x$ will get *more attention* via a *larger attention weight* assigned to the key's corresponding value $y_i$

# 5.2 Attention pooling: Nadaraya-Watson kernel regression

- notably, Nadaraya-Watson kernel regression is a nonparametric model; thus (4) is an example of *nonparametric attention pooling*
- nonparametric Nadaraya-Watson kernel regression enjoys the *consistency* benefit: given enough data, this model converges to the optimal solution
- nonetheless, we can easily integrate learnable parameters into attention pooling
- as an example, slightly different from (4), in the following, the distance between the query $x$ and the key $x_i$ is multiplied by a learnable parameter $w$:

$$
\begin{aligned}
f(x) &= \sum_{i=1}^{n} \alpha(x, x_i) y_i \\
&= \sum_{i=1}^{n} \frac{\exp\left(-\frac{1}{2}((x - x_i)w)^2\right)}{\sum_{j=1}^{n} \exp\left(-\frac{1}{2}((x - x_j)w)^2\right)} y_i \\
&= \sum_{i=1}^{n} \operatorname{softmax}\left(-\frac{1}{2}((x - x_i)w)^2\right) y_i.
\end{aligned}
$$

# 5.3 Attention scoring functions

- in the previous section, we used a Gaussian kernel to model interactions between queries and keys
- treating the exponent of the Gaussian kernel in (4) as an *attention scoring function* (or *scoring function*, for short), the results of this function were essentially fed into a softmax operation
- as a result, we obtained a probability distribution (attention weights) over values that are paired with keys
- in the end, the output of the attention pooling is simply a weighted sum of the values based on these attention weights

# 5.3 Attention scoring functions

- at a high level, we can use the above algorithm to instantiate the framework of attention mechanisms in Figure 3
- denoting an attention scoring function by $a$, Figure 4 illustrates how the output of attention pooling can be computed as a weighted sum of values
- since attention weights are a probability distribution, the weighted sum is essentially a weighted average

# 5.3 Attention scoring functions

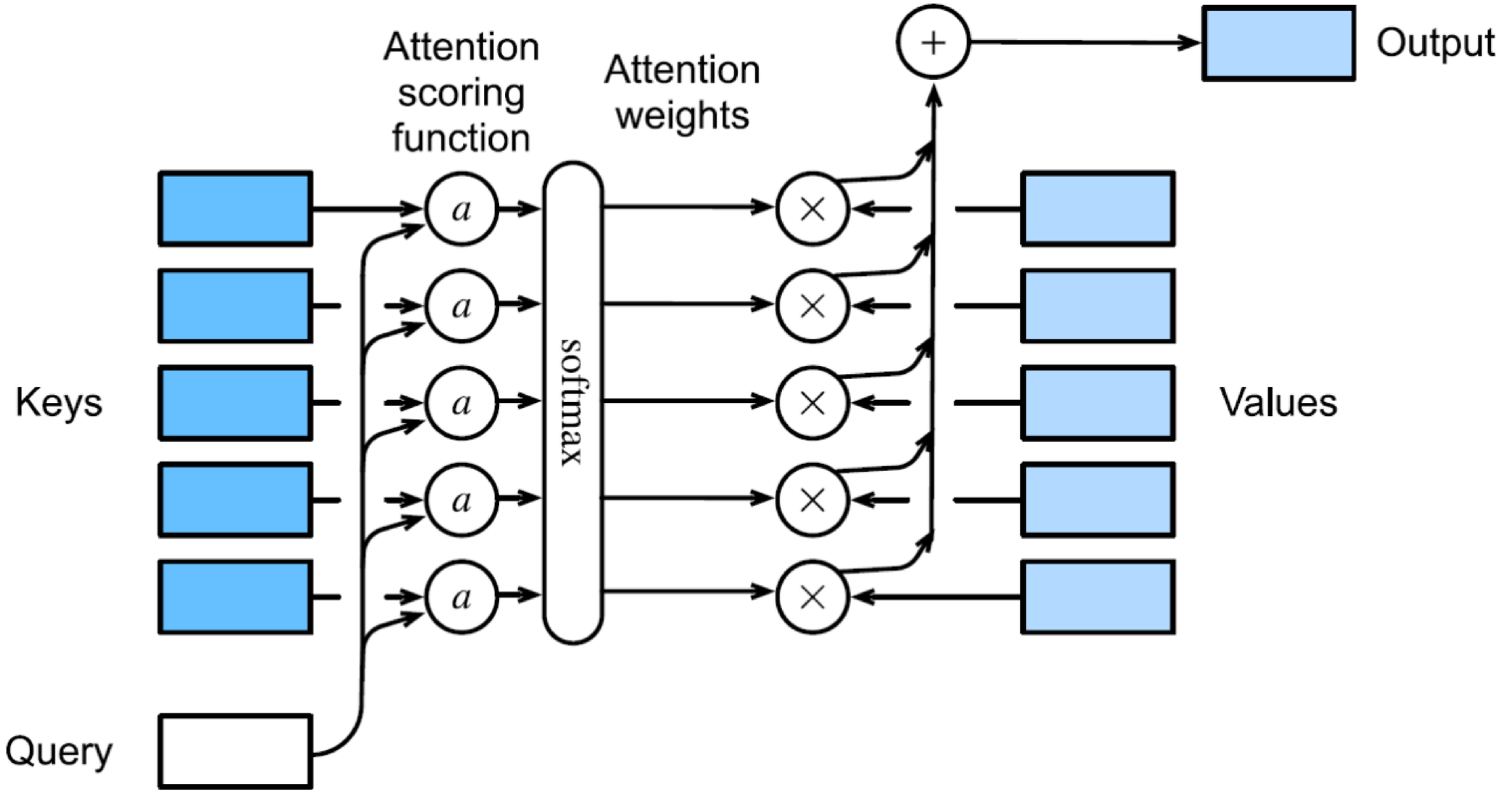



Figure 4: Computing the output of attention pooling as a weighted average of values.

# 5.3 Attention scoring functions

- mathematically, suppose that we have a query $\boldsymbol{q} \in \mathbb{R}^q$ and $m$ key-value pairs $(\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)$, where any $\boldsymbol{k}_i \in \mathbb{R}^k$ and any $\boldsymbol{v}_i \in \mathbb{R}^v$
- the attention pooling $f$ is instantiated as a weighted sum of the values:

$$f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)) = \sum_{i=1}^{m} \alpha(\boldsymbol{q}, \boldsymbol{k}_i) \boldsymbol{v}_i \in \mathbb{R}^v, \tag{5}$$

where the attention weight (scalar) for the query $\boldsymbol{q}$ and key $\boldsymbol{k}_i$ is computed by the softmax operation of an attention scoring function $a$ that maps two vectors to a scalar:

$$\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = \text{softmax}(a(\boldsymbol{q}, \boldsymbol{k}_i)) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_j))} \in \mathbb{R}. \tag{6}$$

- as we can see, different choices of the attention scoring function $a$ lead to different behaviors of attention pooling
- in this section, we introduce two popular scoring functions that we will use to develop more sophisticated attention mechanisms later

# 5.3 Attention scoring functions

- in general, when queries and keys are vectors of different lengths, we can use additive attention as the scoring function
- given a query $\boldsymbol{q} \in \mathbb{R}^q$ and a key $\boldsymbol{k} \in \mathbb{R}^k$, the *additive attention* scoring function is defined as:

$$a(\boldsymbol{q}, \boldsymbol{k}) = \boldsymbol{w}_v^\top \tanh(\boldsymbol{W}_q \boldsymbol{q} + \boldsymbol{W}_k \boldsymbol{k}) \in \mathbb{R}, \tag{7}$$

  where the parameters $\boldsymbol{W}_q \in \mathbb{R}^{h \times q}$, $\boldsymbol{W}_k \in \mathbb{R}^{h \times k}$, and $\boldsymbol{w}_v \in \mathbb{R}^h$ are learnable
- equivalent to (7), the query and the key are concatenated and fed into an MLP with a single hidden layer whose number of hidden units is $h$, a hyperparameter, and is implemented by using tanh as the activation function and disabling bias terms

# 5.3 Attention scoring functions

- a more computationally efficient design for the scoring function can be simply the *dot product*
- however, the dot product operation requires that both the query and the key have the same vector length, say $d$
- assume that all the elements of the query and the key are independent random variables with zero mean and unit variance
- the dot product of both vectors has zero mean and a variance of $d$

# 5.3 Attention scoring functions

- to ensure that the variance of the dot product still remains one, regardless of vector length, the *scaled dot-product attention* scoring function:

$$a(\boldsymbol{q}, \boldsymbol{k}) = \boldsymbol{q}^\top \boldsymbol{k} / \sqrt{d}$$

  divides the dot product by $\sqrt{d}$
- in practice, we often think in mini-batches for efficiency, such as computing attention for $n$ queries and $m$ key-value pairs, where queries and keys are of length $d$, and values are of length $v$
- the scaled dot-product attention of queries $\boldsymbol{Q} \in \mathbb{R}^{n \times d}$, keys $\boldsymbol{K} \in \mathbb{R}^{m \times d}$, and values $\boldsymbol{V} \in \mathbb{R}^{m \times v}$ is:

$$\text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}}\right)\boldsymbol{V} \in \mathbb{R}^{n \times v}. \tag{8}$$

# 5.4 Bahdanau attention

- we studied the machine translation problem in Section 4.6, where we designed an encoder-decoder architecture, based on two RNNs, for sequence to sequence learning
- specifically, the RNN encoder transforms a variable-length sequence into a fixed-shape context variable, then the RNN decoder generates the output (target) sequence token by token, based on the generated tokens and the context variable
- however, even though not all the input (source) tokens are useful for decoding a certain token, the *same* context variable that encodes the entire input sequence is still used at each decoding step

# 5.4 Bahdanau attention

- in a separate but related challenge of handwriting generation for a given text sequence, Graves designed a differentiable attention model to align text characters with the much longer pen trace, where the alignment moves only in one direction
- inspired by the idea of learning to align, Bahdanau et al. proposed a differentiable attention model without the severe unidirectional alignment limitation
- when predicting a token, if not all the input tokens are relevant, the model aligns (or attends) only to parts of the input sequence that are relevant to the current prediction
- this is achieved by treating the context variable as an output of attention pooling

# 5.4 Bahdanau attention

- when describing Bahdanau attention for the RNN encoder-decoder below, we will follow the same notation in Section 4.6
- the new attention-based model is the same as that in Section 4.6, except that the context variable $\boldsymbol{c}$ is replaced by $\boldsymbol{c}_{t'}$ at any decoding time step $t'$
- suppose that there are $T$ tokens in the input sequence, then the context variable at the decoding time step $t'$ is the output of attention pooling:

$$\boldsymbol{c}_{t'} = \sum_{t=1}^{T} \alpha(\boldsymbol{s}_{t'-1}, \boldsymbol{h}_t)\boldsymbol{h}_t,$$

  where the decoder hidden state $\boldsymbol{s}_{t'-1}$ at time step $t'-1$ is the query, and the encoder hidden states $\boldsymbol{h}_t$ are both the keys and values, and the attention weight $\alpha$ is computed as in (6), using the additive attention scoring function defined by (7)
- slightly different from the vanilla RNN encoder-decoder architecture, the same architecture with Bahdanau attention is depicted in Figure 5

# 5.4 Bahdanau attention

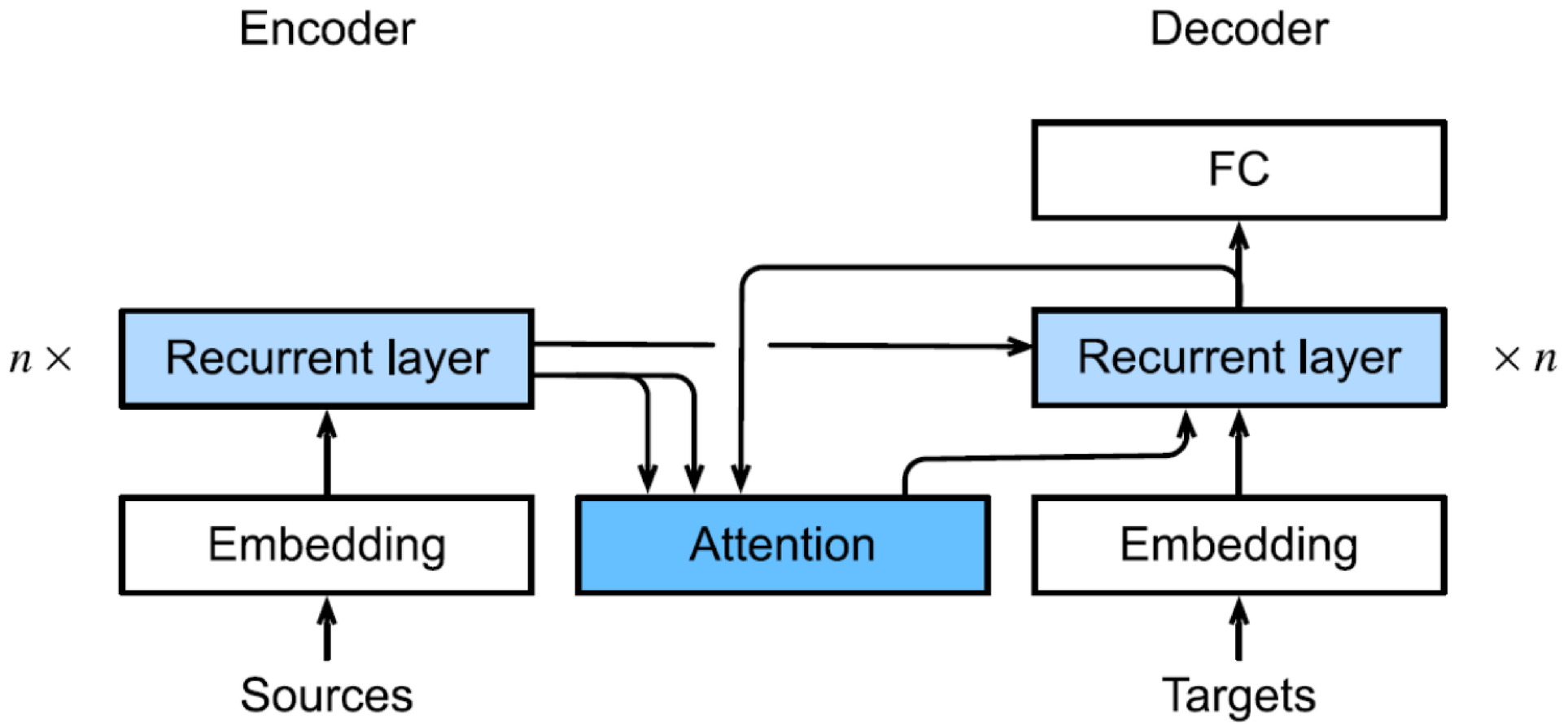



Figure 5: Layers in an RNN encoder-decoder model with Bahdanau attention.

# 5.5 Multi-head attention

- in practice, given the same set of queries, keys, and values, we may want our model to combine knowledge from different behaviors of the same attention mechanism, such as capturing dependencies of various ranges (e.g., shorter-range vs. longer-range) within a sequence
- thus, it may be beneficial to allow our attention mechanism to jointly use different representation subspaces of queries, keys, and values

# 5.5 Multi-head attention

- to this end, instead of performing a single attention pooling, queries, keys, and values can be transformed with *h* independently learned linear projections
- then, these *h* projected queries, keys, and values are fed into attention pooling in parallel
- in the end, the *h* attention pooling outputs are concatenated and transformed with another learned linear projection, to produce the final output
- this design is called *multi-head attention*, where each of the *h* attention pooling outputs is a *head*
- using fully-connected layers to perform learnable linear transformations, Figure 6 describes multi-head attention

# 5.5 Multi-head attention

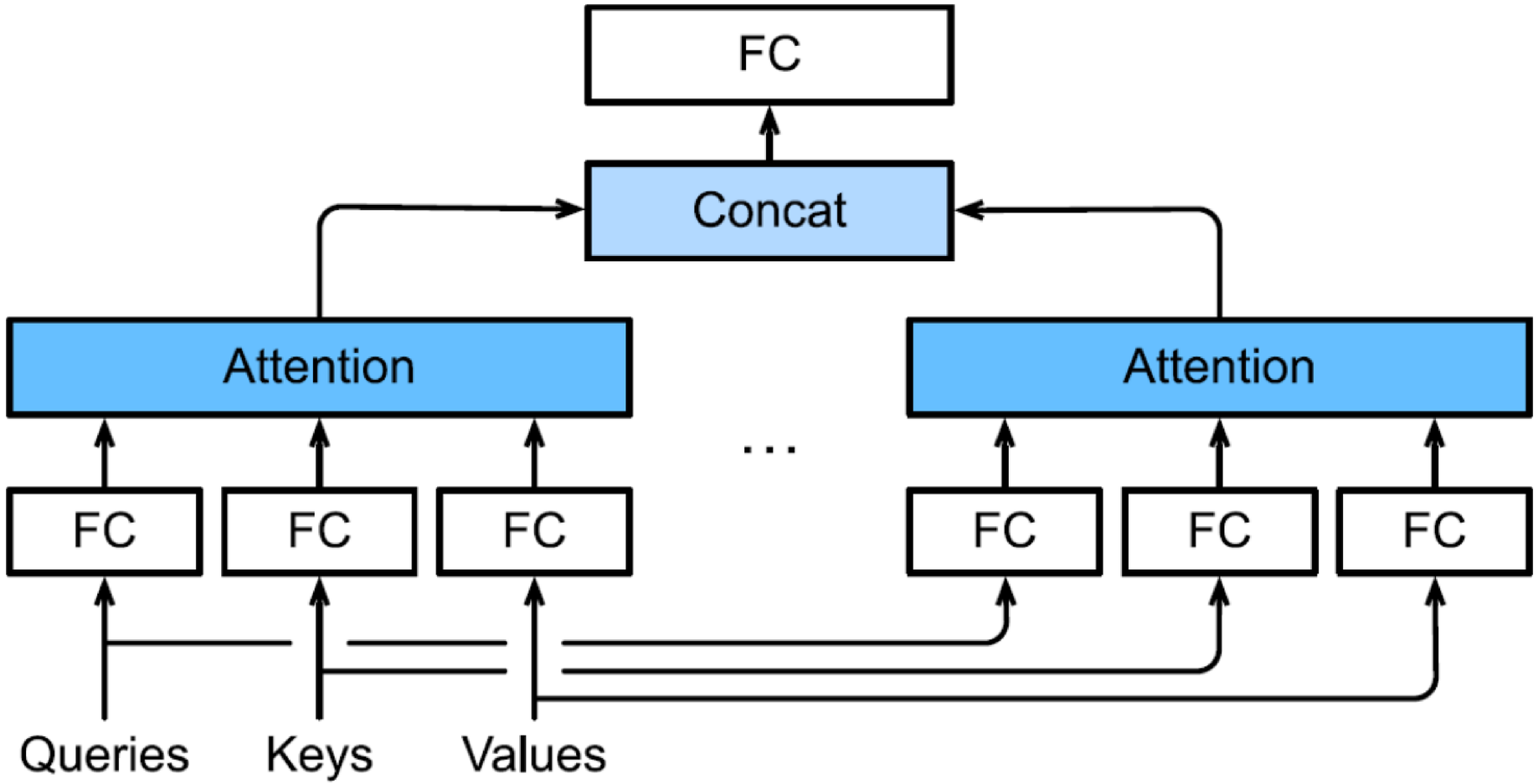



Figure 6: Multi-head attention, where multiple heads are concatenated, then linearly transformed.

# 5.5 Multi-head attention

- let us formalize this model mathematically; given a query $\boldsymbol{q} \in \mathbb{R}^{d_q}$ , a key $\boldsymbol{k} \in \mathbb{R}^{d_k}$, and a value $\boldsymbol{v} \in \mathbb{R}^{d_v}$, each attention head $\boldsymbol{h}_i$ ($i = 1, \ldots, h$) is computed as:

$$\boldsymbol{h}_i = f(\boldsymbol{W}_i^{(q)}\boldsymbol{q}, \boldsymbol{W}_i^{(k)}\boldsymbol{k}, \boldsymbol{W}_i^{(v)}\boldsymbol{v}) \in \mathbb{R}^{p_v},$$

  where the parameters $\boldsymbol{W}_i^{(q)} \in \mathbb{R}^{p_q \times d_q}$, $\boldsymbol{W}_i^{(k)} \in \mathbb{R}^{p_k \times d_k}$, and $\boldsymbol{W}_i^{(v)} \in \mathbb{R}^{p_v \times d_v}$ are learnable, and $f$ is attention pooling, such as additive attention or scaled dot-product attention, discussed in Section 5.3
- the multi-head attention output is another linear transformation via learnable parameters $\boldsymbol{W}_o \in \mathbb{R}^{p_o \times hp_v}$ of the concatenation of the $h$ heads:

$$\boldsymbol{W}_o \begin{bmatrix} \boldsymbol{h}_1 \\ \vdots \\ \boldsymbol{h}_h \end{bmatrix} \in \mathbb{R}^{p_o}.$$

- based on this design, each head may attend to different parts of the input; more sophisticated functions than the simple weighted average can be expressed

# 5.6 Self-attention and positional encoding

- in deep learning, we often use CNNs or RNNs to encode a sequence
- now, with attention mechanisms, imagine that we feed a sequence of tokens into attention pooling, so that the same set of tokens act as queries, keys, and values
- specifically, each query attends to all the key-value pairs and generates one attention output
- since the queries, keys, and values come from the same place, this performs *self-attention*, which is also called *intra-attention*
- in this section, we will discuss sequence encoding using self-attention, including using additional information for the sequence order

# 5.6 Self-attention and positional encoding

- given a sequence of input tokens $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$, where any $\boldsymbol{x}_i \in \mathbb{R}^d$ $(1 \leq i \leq n)$, its self-attention outputs a sequence of the same length $\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n$, where:

$$\boldsymbol{y}_i = f(\boldsymbol{x}_i, (\boldsymbol{x}_1, \boldsymbol{x}_1), \ldots, (\boldsymbol{x}_n, \boldsymbol{x}_n)) \in \mathbb{R}^d,$$

  according to the definition of attention pooling $f$ in (5)
- let us compare architectures for mapping a sequence of $n$ tokens to another sequence of equal length, where each input or output token is represented by a $d$-dimensional vector
- specifically, we will consider CNNs, RNNs, and self-attention
- we will compare their computational complexity, sequential operations, and maximum path lengths
- note that sequential operations prevent parallel computation, while a shorter path between any combination of sequence positions makes it easier to learn long-range dependencies within the sequence

# 5.6 Self-attention and positional encoding

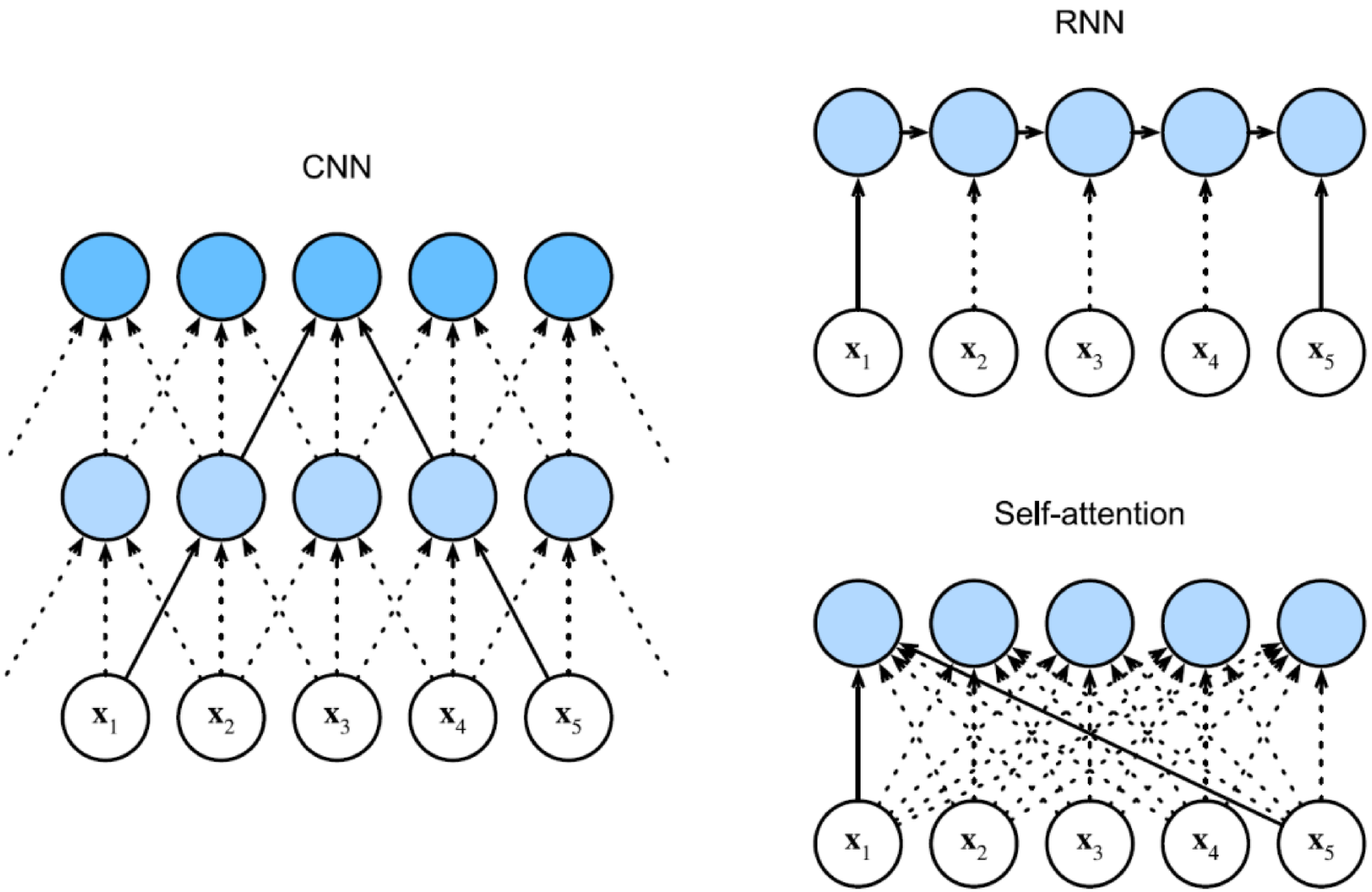



Figure 7: Comparing CNN (padding tokens are omitted), RNN, and self-attention architectures.

# 5.6 Self-attention and positional encoding

- consider a convolutional layer whose kernel size is $k$
- since the sequence length is $n$, the numbers of input and output channels are both $d$, the computational complexity of the convolutional layer is $\mathcal{O}(knd^2)$
- as Figure 7 shows, CNNs are hierarchical, so there are $\mathcal{O}(1)$ sequential operations and the maximum path length is $\mathcal{O}(n/k)$
- for example, $\boldsymbol{x}_1$ and $\boldsymbol{x}_5$ are within the receptive field of a two-layer CNN with kernel size 3 in Figure 7

# 5.6 Self-attention and positional encoding

- when updating the hidden state of RNNs, multiplication of the $d \times d$ weight matrix and the $d$-dimensional hidden state has a computational complexity of $\mathcal{O}(d^2)$
- since the sequence length is $n$, the computational complexity of the recurrent layer is $\mathcal{O}(nd^2)$
- according to Figure 7, there are $\mathcal{O}(n)$ sequential operations that cannot be parallelized and the maximum path length is also $\mathcal{O}(n)$

# 5.6 Self-attention and positional encoding

- in self-attention, the queries, keys, and values are all $n \times d$ matrices
- consider the scaled dot-product attention in (8), where a $n \times d$ matrix is multiplied by a $d \times n$ matrix, then the output $n \times n$ matrix is multiplied by a $n \times d$ matrix
- as a result, the self-attention has a $\mathcal{O}(n^2 d)$ computational complexity
- as we can see in Figure 7, each token is directly connected to any other token via self-attention
- therefore, computation can be parallel with $\mathcal{O}(1)$ sequential operations and the maximum path length is also $\mathcal{O}(1)$

# 5.6 Self-attention and positional encoding

- all in all, both CNNs and self-attention enjoy parallel computation, and self-attention has the shortest maximum path length
- however, the quadratic computational complexity with respect to the sequence length makes self-attention prohibitively slow for very long sequences
- unlike RNNs that recurrently process tokens of a sequence one by one, self-attention ditches sequential operations in favor of parallel computation
- to use the sequence order information, we can inject absolute or relative positional information by adding *positional encoding* to the input representations
- positional encodings can be either learned or fixed; in the following, we describe a fixed positional encoding based on sine and cosine functions

# 5.6 Self-attention and positional encoding

- suppose that the input representation $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ contains the $d$-dimensional embeddings for $n$ tokens of a sequence
- the positional encoding outputs $\boldsymbol{X} + \boldsymbol{P}$ using a positional embedding matrix $\boldsymbol{P} \in \mathbb{R}^{n \times d}$ of the same shape, whose element on the $i$th row and the $(2j)$th or the $(2j+1)$th column is:

$$
\begin{aligned}
p_{i,2j} &= \sin\left(\frac{i}{10000^{2j/d}}\right), \\
p_{i,2j+1} &= \cos\left(\frac{i}{10000^{2j/d}}\right).
\end{aligned} \tag{9}
$$

# 5.6 Self-attention and positional encoding

- in the positional embedding matrix $\boldsymbol{P}$, rows correspond to positions within a sequence and columns represent different positional encoding dimensions
- in the example in Figure 8, we can see that the 6th and the 7th columns of the positional embedding matrix have a higher frequency than the 8th and the 9th columns
- the offset between the 6th and the 7th (same for the 8th and the 9th) columns is due to the alternation of sine and cosine functions

# 5.6 Self-attention and positional encoding

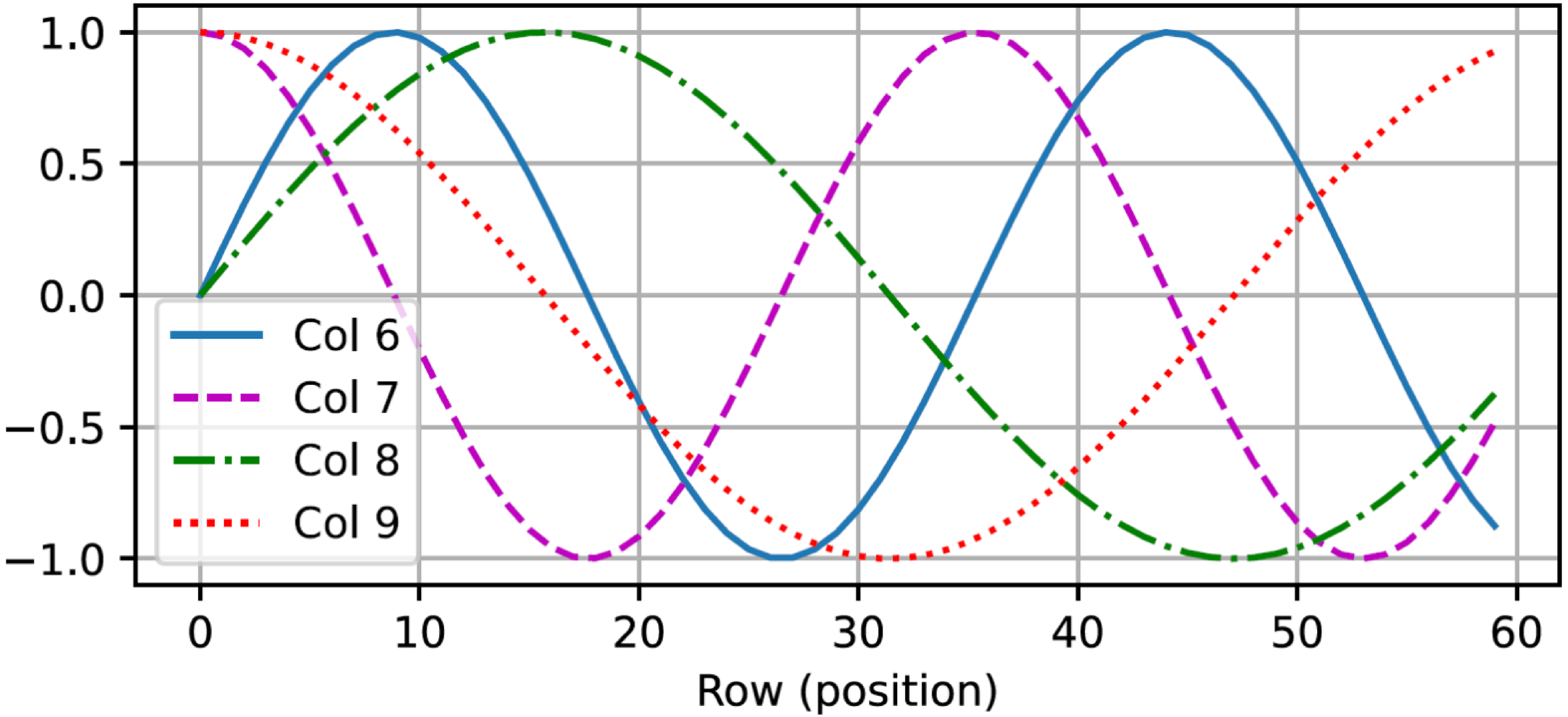



Figure 8: Positional encoding example.

# 5.6 Self-attention and positional encoding

- besides capturing absolute positional information, the above positional encoding also allows a model to easily learn to attend by relative positions
- this is because, for any fixed position offset $\delta$, the positional encoding at position $i + \delta$ can be represented by a linear projection of that at position $i$
- this projection can be explained mathematically; denoting $\omega_j = 1/10000^{2j/d}$, any pair of $(p_{i,2j}, p_{i,2j+1})$ in (9) can be linearly projected to $(p_{i+\delta,2j}, p_{i+\delta,2j+1})$, for any fixed offset $\delta$:

$$\begin{aligned}
\begin{bmatrix} \cos(\delta\omega_j) & \sin(\delta\omega_j) \\ -\sin(\delta\omega_j) & \cos(\delta\omega_j) \end{bmatrix} \begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix} &= \begin{bmatrix} \cos(\delta\omega_j)\sin(i\omega_j) + \sin(\delta\omega_j)\cos(i\omega_j) \\ -\sin(\delta\omega_j)\sin(i\omega_j) + \cos(\delta\omega_j)\cos(i\omega_j) \end{bmatrix} \\
&= \begin{bmatrix} \sin((i+\delta)\omega_j) \\ \cos((i+\delta)\omega_j) \end{bmatrix} \\
&= \begin{bmatrix} p_{i+\delta,2j} \\ p_{i+\delta,2j+1} \end{bmatrix},
\end{aligned}$$

where the $2 \times 2$ projection matrix does not depend on any position index $i$

# 5.7 Transformer

- we have compared CNNs, RNNs, and self-attention in Section 5.6
- notably, self-attention enjoys both parallel computation and the shortest maximum path length
- therefore, natually, it is appealing to design deep architectures by using self-attention
- unlike earlier self-attention models that still rely on RNNs for input representations, the *transformer* model is solely based on attention mechanisms, without any convolutional or recurrent layer

# 5.7 Transformer

- though originally proposed for sequence to sequence learning on text data, transformers have been pervasive in a wide range of modern deep learning applications, such as in areas of language, vision, speech, and reinforcement learning
- as an instance of the encoder-decoder architecture, the overall architecture of the transformer is presented in Figure 9
- as we can see, the transformer is composed of an encoder and a decoder
- different from Bahdanau attention for sequence to sequence learning in Figure 5, the input (source) and output (target) sequence embeddings are added with positional encoding, before being fed into the encoder and the decoder that stack modules based on self-attention

# 5.7 Transformer

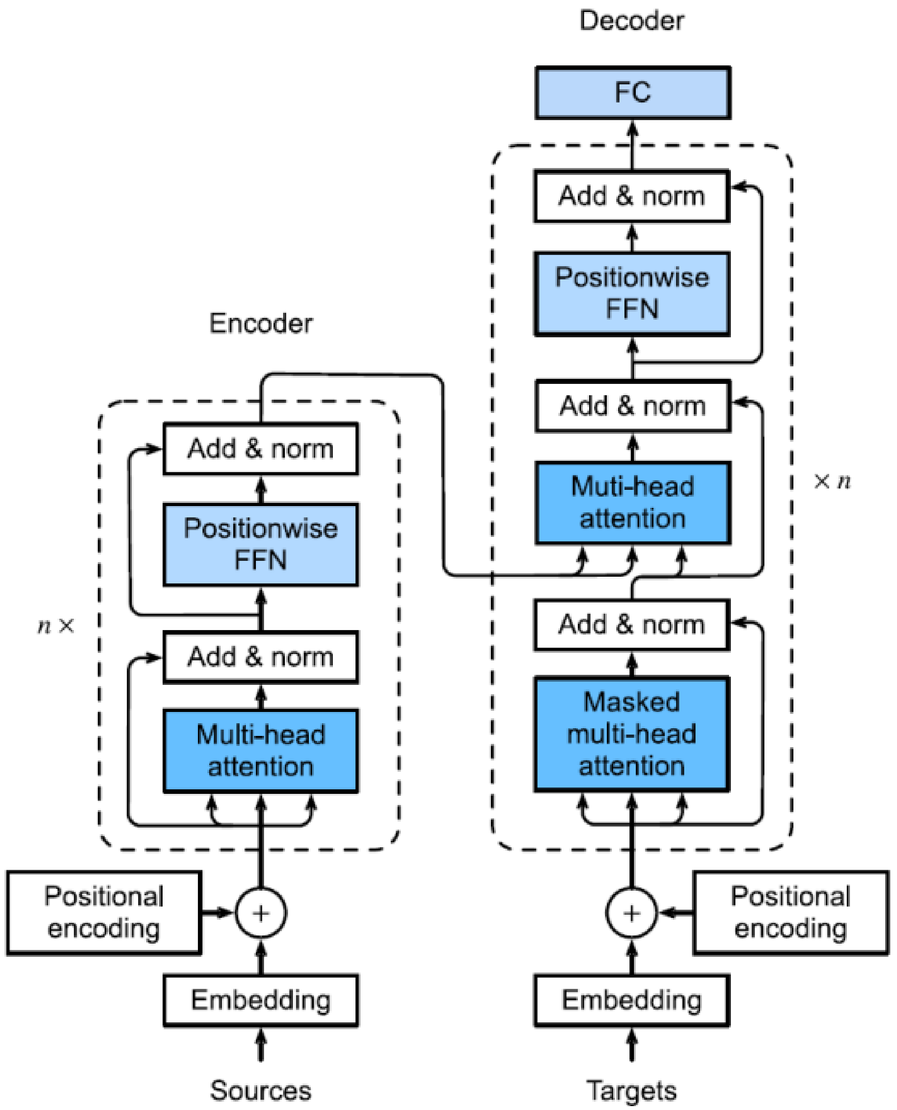



Figure 9: The transformer architecture.

# 5.7 Transformer

- now, we provide an overview of the transformer architecture in Figure 9
- on a high level, the transformer encoder is a stack of multiple identical layers, where each layer has two sublayers (either is denoted as sublayer)
- the first is a multi-head self-attention pooling and the second is a position-wise feed-forward network
- specifically, in the encoder self-attention, queries, keys, and values are all from the outputs of the previous encoder layer

# 5.7 Transformer

- inspired by the ResNet design in Section 3.10, a residual connection is employed around both sublayers
- in the transformer, for any input $\boldsymbol{x} \in \mathbb{R}^d$, at any position of the sequence, we require that sublayer$(\boldsymbol{x}) \in \mathbb{R}^d$, so that the residual connection $\boldsymbol{x} + \text{sublayer}(\boldsymbol{x}) \in \mathbb{R}^d$ is feasible
- this addition from the residual connection is immediately followed by *layer normalization*
- as a result, the transformer encoder outputs a $d$-dimensional vector representation for each position of the input sequence

# 5.7 Transformer

- the transformer decoder is also a stack of multiple identical layers with residual connections and layer normalizations
- besides the two sublayers described in the encoder, the decoder inserts a third sublayer, known as the *encoder-decoder attention*, between these two
- in the encoder-decoder attention, queries are from the outputs of the previous decoder layer, and the keys and values are from the transformer encoder outputs
- in the decoder self-attention, queries, keys, and values are all from the outputs of the previous decoder layer
- however, each position in the decoder is allowed to only attend to all positions in the decoder up to that position
- this *masked* attention preserves the auto-regressive property, ensuring that the prediction only depends on those output tokens that have been generated

# 5.7 Transformer

- the position-wise feed-forward network transforms the representation at all the sequence positions using the same MLP; this is why we call it *position-wise*
- now, let us focus on the “add & norm” component in Figure 9
- as we described at the beginning of this section, this is a residual connection immediately followed by layer normalization; both are key to effective deep architectures
- in Section 3.9, we explained how batch normalization recenters and rescales across the examples within a mini-batch
- layer normalization is the same as batch normalization, except that the former normalizes across the feature dimension
- despite its pervasive applications in computer vision, batch normalization is usually empirically less effective than layer normalization in natural language processing tasks, whose inputs are often variable-length sequences

# 5.7 Transformer

- when training sequence-to-sequence models, tokens at all the positions (time steps) of the output sequence are known
- however, during prediction, the output sequence is generated token by token; thus, at any decoder time step, only the generated tokens can be used in the decoder self-attention
- to preserve auto-regression in the decoder, its masked self-attention specifies that any query should only attend to all positions in the decoder up to the query position
- although the transformer architecture was originally proposed for sequence-to-sequence learning, either the transformer encoder or the transformer decoder is often individually used for different deep learning tasks

# Exercises

## Exercise 5

Given the 3 key-value pairs $\{(\boldsymbol{k}_1, \boldsymbol{v}_1), (\boldsymbol{k}_2, \boldsymbol{v}_2), (\boldsymbol{k}_3, \boldsymbol{v}_3)\} = \{([2,0,1]^\top, [-1,1]^\top), ([0,1,0]^\top, [-1,0]^\top), ([0,2,0]^\top, [0,1]^\top)\}$, and the query $\boldsymbol{q} = [1,2,3]^\top$, compute the output of attention pooling for additive attention, and scale dot-product attention, respectively. The necessary weights are generated randomly having values from the set $\{-1,0,1\}$ (you can generate them yourself, not all zeros).

**Solution:**

- we have the following formulas:

$$f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)) = \sum_{i=1}^{m} \alpha(\boldsymbol{q}, \boldsymbol{k}_i)\boldsymbol{v}_i \in \mathbb{R}^v,$$

$$\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = \text{softmax}(a(\boldsymbol{q}, \boldsymbol{k}_i)) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_j))} \in \mathbb{R},$$

where the query $\boldsymbol{q} \in \mathbb{R}^q$, the $m$ key-value pairs $(\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)$, where any $\boldsymbol{k}_i \in \mathbb{R}^k$ and any $\boldsymbol{v}_i \in \mathbb{R}^v$

# Exercises

- *additive attention*:

$$a(\boldsymbol{q}, \boldsymbol{k}) = \boldsymbol{w}_v^\top \tanh(\boldsymbol{W}_q \boldsymbol{q} + \boldsymbol{W}_k \boldsymbol{k}) \in \mathbb{R},$$

  where the parameters are $\boldsymbol{W}_q \in \mathbb{R}^{h \times q}$, $\boldsymbol{W}_k \in \mathbb{R}^{h \times k}$, and $\boldsymbol{w}_v \in \mathbb{R}^h$
- *scaled dot-product attention*:

$$a(\boldsymbol{q}, \boldsymbol{k}) = \boldsymbol{q}^\top \boldsymbol{k} / \sqrt{d} \in \mathbb{R},$$

  where $q = k = d$
- in our case, $m = 3$, $k = 3$, $v = 2$, $q = 3$
- we take $h = 2$ and

$$\boldsymbol{w}_v = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \in \mathbb{R}^2,$$

$$\boldsymbol{W}_q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 3},$$

$$\boldsymbol{W}_k = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix} \in \mathbb{R}^{2 \times 3}.$$

# Exercises

- we apply the formula for additive attention:

$$
\begin{aligned}
a(\boldsymbol{q}, \boldsymbol{k}_1) &= \begin{bmatrix} 1 & 0 \end{bmatrix} \tanh\left( \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix} \right) \\
&= \begin{bmatrix} 1 & 0 \end{bmatrix} \tanh\left( \begin{bmatrix} 4 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right) \\
&= \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \tanh 4 \\ \tanh 2 \end{bmatrix} \\
&= \tanh 4 \in \mathbb{R},
\end{aligned}
$$

# Exercises

$$
\begin{aligned}
a(\boldsymbol{q}, \boldsymbol{k}_2) &= \begin{bmatrix} 1 & 0 \end{bmatrix} \tanh \left( \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \right) \\
&= \begin{bmatrix} 1 & 0 \end{bmatrix} \tanh \left( \begin{bmatrix} 4 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) \\
&= \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \tanh 5 \\ \tanh 2 \end{bmatrix} \\
&= \tanh 5 \in \mathbb{R},
\end{aligned}
$$

# Exercises

$$
\begin{aligned}
a(\boldsymbol{q}, \boldsymbol{k}_3) &= \begin{bmatrix} 1 & 0 \end{bmatrix} \tanh \left( \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix} \right) \\
&= \begin{bmatrix} 1 & 0 \end{bmatrix} \tanh \left( \begin{bmatrix} 4 \\ 2 \end{bmatrix} + \begin{bmatrix} 2 \\ 0 \end{bmatrix} \right) \\
&= \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \tanh 6 \\ \tanh 2 \end{bmatrix} \\
&= \tanh 6 \in \mathbb{R},
\end{aligned}
$$

# Exercises

$$
\begin{aligned}
f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), (\boldsymbol{k}_2, \boldsymbol{v}_2), (\boldsymbol{k}_3, \boldsymbol{v}_3)) &= \frac{\exp(\tanh 4)}{\exp(\tanh 4) + \exp(\tanh 5) + \exp(\tanh 6)} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \\
&\quad + \frac{\exp(\tanh 5)}{\exp(\tanh 4) + \exp(\tanh 5) + \exp(\tanh 6)} \begin{bmatrix} -1 \\ 0 \end{bmatrix} \\
&\quad + \frac{\exp(\tanh 6)}{\exp(\tanh 4) + \exp(\tanh 5) + \exp(\tanh 6)} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\
&= \frac{1}{\exp(\tanh 4) + \exp(\tanh 5) + \exp(\tanh 6)} \\
&\quad \times \begin{bmatrix} -\exp(\tanh 4) - \exp(\tanh 5) \\ \exp(\tanh 4) + \exp(\tanh 6) \end{bmatrix} \in \mathbb{R}^2.
\end{aligned}
$$

# Exercises

- now, we apply the formula for scaled dot-product attention:

$$a(\boldsymbol{q}, \boldsymbol{k}_1) = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix} = \frac{5}{\sqrt{3}} \in \mathbb{R},$$

$$a(\boldsymbol{q}, \boldsymbol{k}_2) = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \frac{2}{\sqrt{3}} \in \mathbb{R},$$

$$a(\boldsymbol{q}, \boldsymbol{k}_3) = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix} = \frac{4}{\sqrt{3}} \in \mathbb{R},$$

# Exercises

$$
\begin{aligned}
f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), (\boldsymbol{k}_2, \boldsymbol{v}_2), (\boldsymbol{k}_3, \boldsymbol{v}_3)) &= \frac{\exp\left(\frac{5}{\sqrt{3}}\right)}{\exp\left(\frac{5}{\sqrt{3}}\right) + \exp\left(\frac{2}{\sqrt{3}}\right) + \exp\left(\frac{4}{\sqrt{3}}\right)} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \\
&\quad + \frac{\exp\left(\frac{2}{\sqrt{3}}\right)}{\exp\left(\frac{5}{\sqrt{3}}\right) + \exp\left(\frac{2}{\sqrt{3}}\right) + \exp\left(\frac{4}{\sqrt{3}}\right)} \begin{bmatrix} -1 \\ 0 \end{bmatrix} \\
&\quad + \frac{\exp\left(\frac{4}{\sqrt{3}}\right)}{\exp\left(\frac{5}{\sqrt{3}}\right) + \exp\left(\frac{2}{\sqrt{3}}\right) + \exp\left(\frac{4}{\sqrt{3}}\right)} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\
&= \frac{1}{\exp\left(\frac{5}{\sqrt{3}}\right) + \exp\left(\frac{2}{\sqrt{3}}\right) + \exp\left(\frac{4}{\sqrt{3}}\right)} \\
&\quad \times \begin{bmatrix} -\exp\left(\frac{5}{\sqrt{3}}\right) - \exp\left(\frac{2}{\sqrt{3}}\right) \\ \exp\left(\frac{5}{\sqrt{3}}\right) + \exp\left(\frac{4}{\sqrt{3}}\right) \end{bmatrix} \in \mathbb{R}^2.
\end{aligned}
$$

Vă mulţumesc!